merged_v4 상태 머신

`boyun/merged_v4/src`의 컨트롤러 4개를 상태 머신 다이어그램으로 그렸습니다. 상태 안에는 그 상태에서 항상 나오는 출력(Moore)을 적었고, 화살표에는 `조건 / 그때만 나오는 출력(Mealy)`을 적었습니다. 버튼은 디바운서를 거친 1클럭 펄스이고, `rst`는 비동기 리셋입니다.

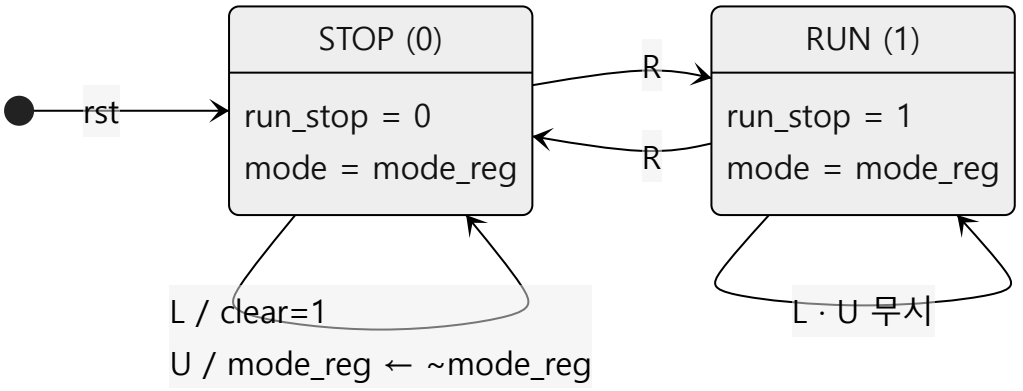
R L U D = btnR · btnL · btnU · btnD **A / x=1** = A일 때 이동하면서 그 클럭에 x=1 ← = 레지스터에 저장

버튼이 가는 곳

Top(`stopwatch_watch.v`)이 스위치로 컨트롤러 하나를 골라 버튼 4개를 AND로 넘깁니다. 고르지 않은 컨트롤러는 버튼이 모두 0으로 보입니다.

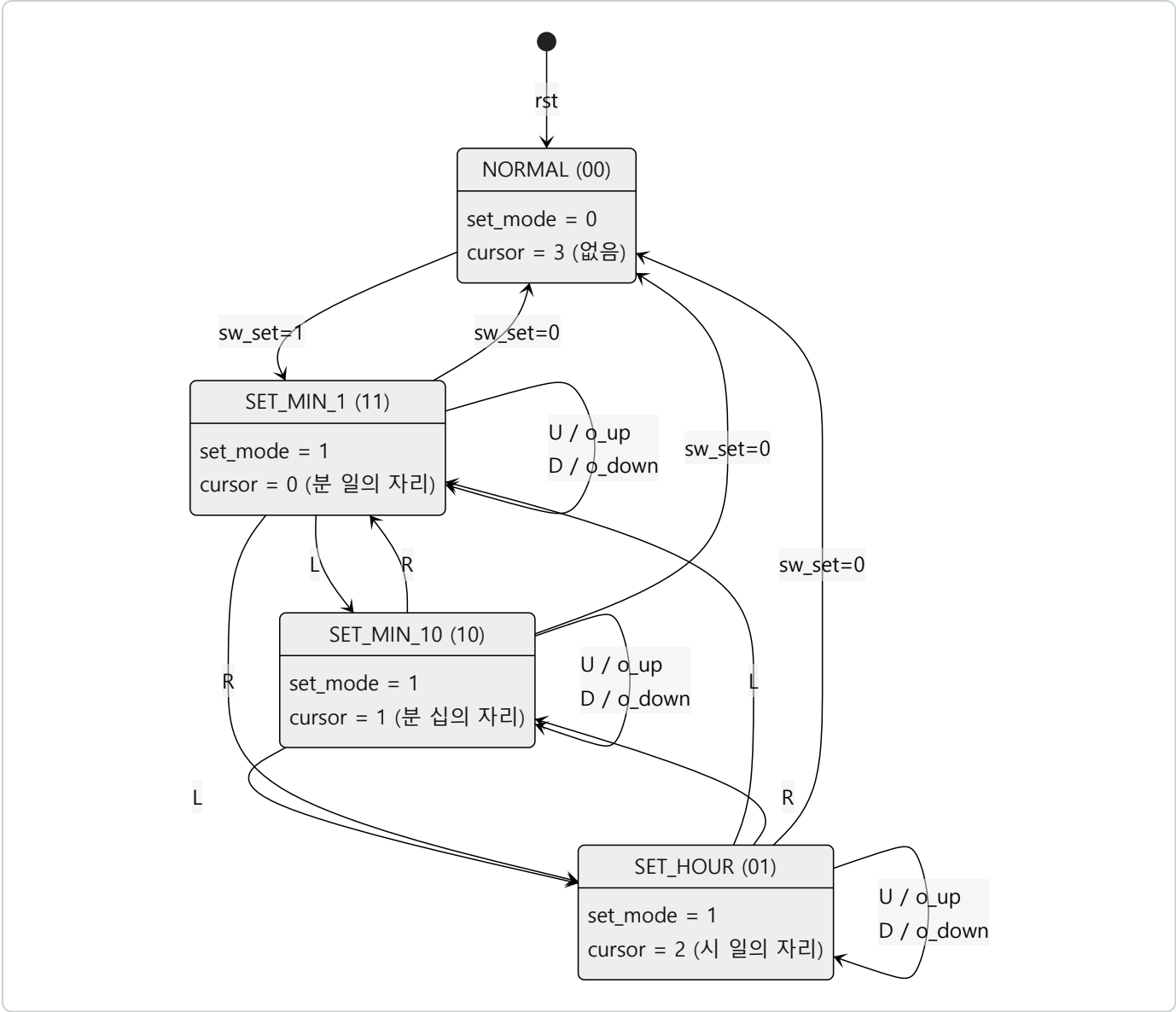
버튼	스톱워치 (sw4=0, sw0=1)	시계 (sw4=0, sw0=0)	타이머 (sw4=1)
R	btn_run_stop: 시작/정지	btn_right: 오른쪽 자리	+1분
L	btn_clear: 지우기 (STOP에서만)	btn_left: 왼쪽 자리	시작 / 취소
U	btn_mode: 업/다운 (STOP에서만)	btn_up: 지금 자리 +1	+10초
D	btn_lap: 랩타임 (lap_controller)	btn_down: 지금 자리 −1	+10분

스톱워치 `stopwatch_control_unit.sv`



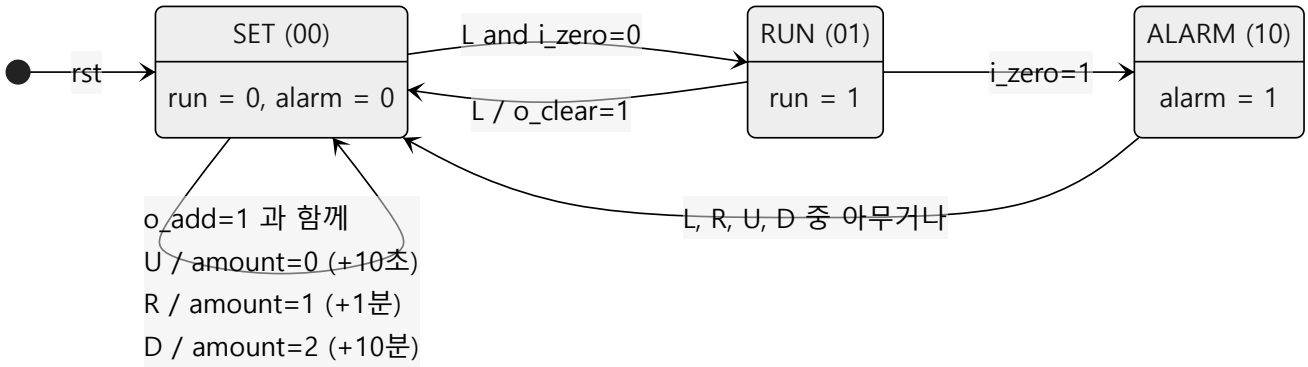
- `clear`는 STOP에서 `btn_clear`를 그대로 내보냅니다. 이 신호가 datapath 카운터와 `lap_controller`를 함께 지웁니다.
- `mode_reg`는 상태와 따로 있는 1비트 레지스터입니다. STOP에서 U가 오면 뒤집히고, 리셋하면 0(업카운트)입니다.
- **남은 차이:** STOP에서 R과 L이 같은 클럭에 오면 RUN으로 가면서 `clear=1`도 나가서 0부터 다시 셉니다.

시계 `watch_control_unit.sv`



- 설정에 들어가면 항상 맨 오른쪽 자리(`SET_MIN_1`)부터 시작합니다. L/R은 세 자리를 돌아가며 움직이고 양 끝에서 반대쪽 끝으로 넘어갑니다.
- 우선순위: `sw_set=0` → L → R. 스위치를 내리면 버튼과 상관없이 NORMAL로 갑니다.
- `o_up` · `o_down`은 이동과 따로 계산합니다. 설정 상태이고 `sw_set=1`이면 L/R로 이동하는 클럭에도 나갈 수 있습니다. U와 D가 같은 클럭에 오면 U가 우선입니다.
- `SET_MIN_10`에서 U/D는 분을 ±10 합니다.

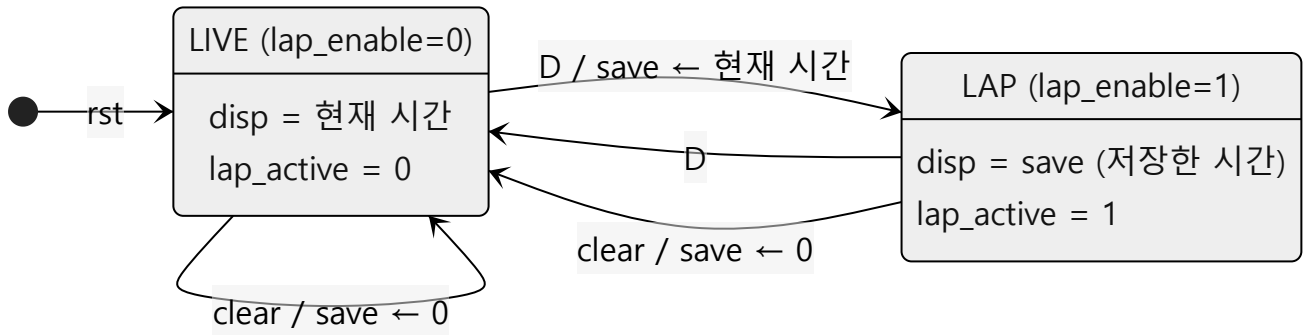
타이머 `timer_control_unit.sv`



- SET 우선순위: L → U → R → D. 남은 시간이 0일 때(`i_zero=1`) L을 누르면 아무 일도 없이 SET에 남습니다.
- RUN 우선순위: L(취소) → `i_zero`. 취소하면 `o_clear`로 남은 시간을 0으로 만듭니다.
- `i_zero`는 `timer_datapath`에서 들어옵니다. 더한 값이 60분을 넘으면 datapath가 59:59로 막습니다.
- ALARM에서 Top이 LED 16개를 0.5초 간격으로 깜빡입니다.

랩타임 `lap_controller.v`

상태를 `enum`으로 쓰지는 않았지만 1비트 레지스터 `lap_enable`이 2상태 머신 역할을 합니다. 아래 LIVE·LAP은 설명하려고 붙인 이름입니다.



- `clear`는 스톱워치 컨트롤러 출력이라 STOP에서 L을 눌렀을 때만 옵니다. `clear`가 D보다 우선입니다.
- D는 스톱워치가 RUN이든 STOP이든 받습니다. LAP에 있는 동안에도 카운터는 계속 돌고, 화면만 저장한 시간을 보여 줍니다.
- LAP일 때 Top이 `led[7]`을 켭니다.

출처: boyun/merged_v4/src (커밋 f39f97e) · 2026-10-16